IA Aplicada com LLMs

Aula 09 — Frameworks e orquestração

Antes disso, uma confusão a desfazer

Camada	Produto	O problema que resolve
Biblioteca	**LangChain**	abstrações sobre modelo, ferramenta e laço
Runtime	**LangGraph**	execução durável, persistência, humano no laço
Plataforma	**LangSmith**	*trace*, avaliação, registro de prompt

São **três produtos distintos**, e quase todo mundo chega tratando os três como um nome só.

Por que a separação importa

As três se adotam SEPARADAMENTE. É possível usar LangGraph sem LangChain. É
possível instrumentar um agente escrito à mão com LangSmith.

> Tratar as três como pacote único produz a decisão de adoção mais cara possível: a de **tudo ou nada**.

A camada que mais frequentemente justifica a adoção sozinha é a do **runtime**: reimplementar execução durável é caro, e é requisito de produção.

O mapa, sobre um eixo só

Framework	Laço	Runtime durável	Modelo mental
LangGraph	sim	sim	grafo de estado com arestas condicionais
CrewAI	sim	parcial	equipe de papéis com tarefas
OpenAI Agents SDK	sim	não	agentes com *handoff*
Google ADK	sim	sim	agentes componíveis
AutoGen	sim	não	conversa entre agentes

Nenhuma linha é elogio ou crítica. São **recortes diferentes**.

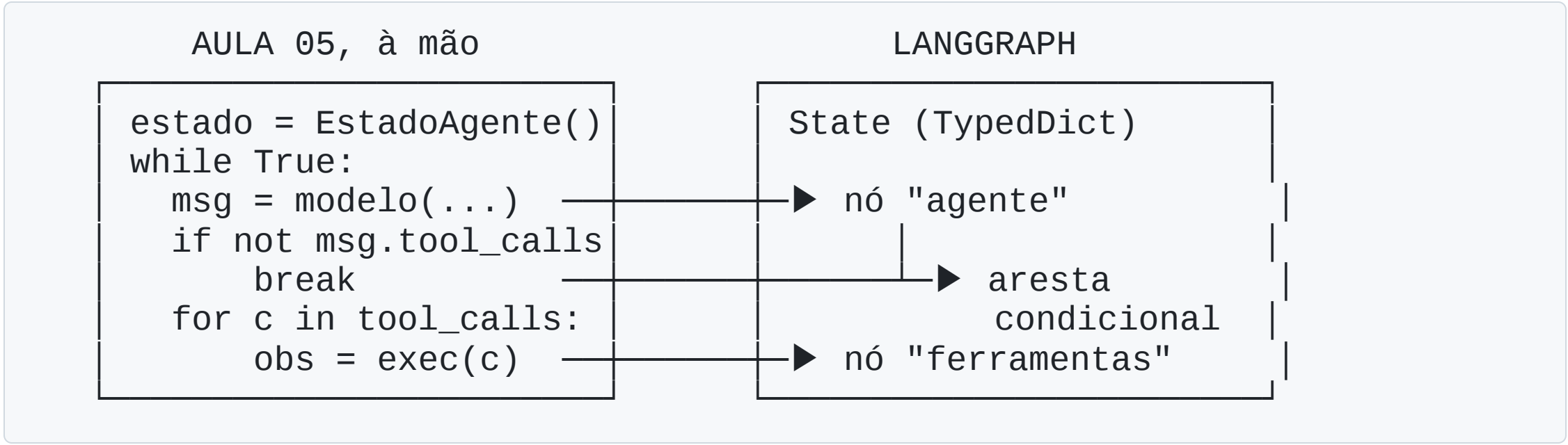
Por que LangGraph, aqui

Externo: a Parte 2 do trabalho exige o grafo em LangChain e LangGraph.

Pedagógico, e é o que importa: o modelo mental de LangGraph é literalmente o que vocês construíram à mão na Aula 05.

> O `EstadoAgente` é o estado. Cada bloco do laço é um nó. O `if` que decide se há `tool_calls` é uma aresta condicional.

O grafo que vocês já escreveram



O `while` vira **topologia**. Mesma ideia, declarada em vez de escrita.

Parte 1

As duas peças de base

O modelo aumentado

Operação	O que ela faz
`with_structured_output(Schema)`	a saída obedece a um **schema**
`bind_tools([...])`	a saída pode ser um **pedido** de chamada

Nenhuma das duas é um agente. **Todos os padrões adiante são combinações delas.**

O modelo NÃO executa nada

Ele devolve **nome e argumentos**. Quem executa é o seu código.

```
Ai Message
Tool Calls:
  multiplicar
  Args:
    a: 2
    b: 3
```

Repare no que não está aí: **o resultado**. É essa separação que o laço fecha.

E o schema garante a FORMA, não o conteúdo

`Literal["conto", "piada", "poema"]` garante que a resposta seja uma das três.

Não garante que seja a certa.

> Pedido: *"me escreva uma piada sobre gatos"*
> Rota escolhida pelo modelo: **conto**
>
> O grafo não quebrou. A decisão estava errada.

Parte 2

Os padrões da Aula 05, como grafo

Nenhum padrão muda. Eles deixam de ser escritos e passam a ser declarados

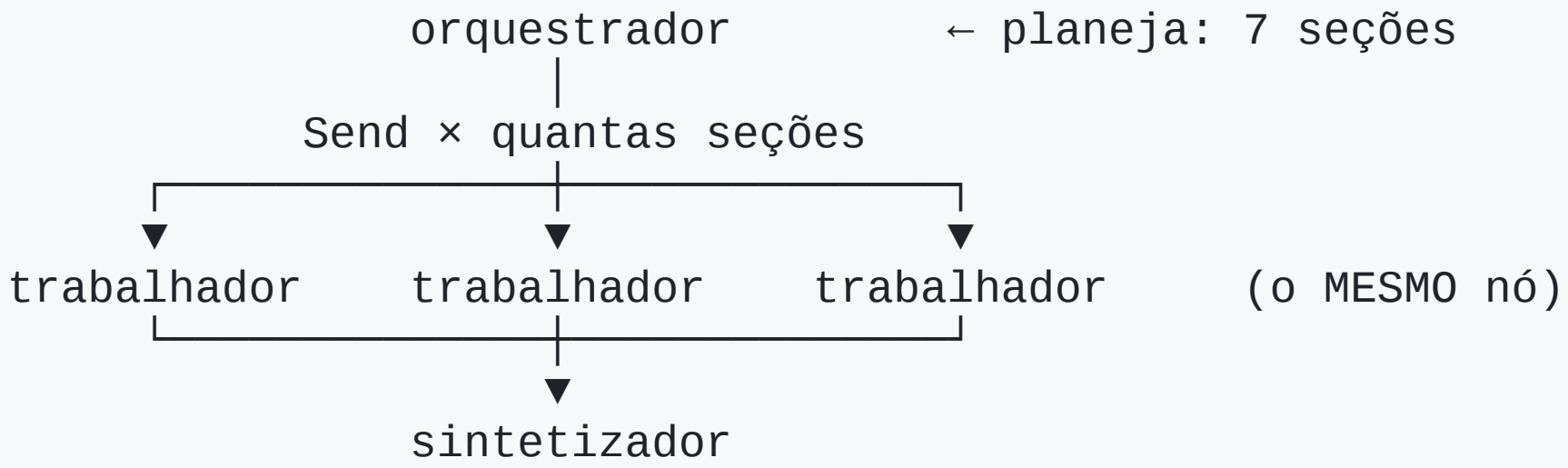
Padrão	Como vira grafo
sequencial com portão	aresta condicional com função em **código puro**
router	aresta condicional decidida pelo **modelo**
paralelização	várias arestas do mesmo ponto + nó agregador
orquestrador-trabalhador	`Send`: um trabalhador **por item**, em runtime
avaliador-otimizador	aresta **de volta**, com contador no estado
agente	aresta condicional enquanto houver `tool_calls`

Quem decide a bifurcação

Degrau	Quando basta	Custo
regra em código	a condição é verificável	zero
embedding	condição de assunto, rotas estáveis	chamada barata
modelo	exige julgamento sobre texto livre	chamada cara, **e que erra**

A cascata da Aula 06 continua valendo: **o degrau mais barato que resolve**.

O único padrão em que o grafo não sabe quantos nós vai executar



O grafo tem **um** nó de trabalhador. Quantas cópias rodam é decisão de runtime.

Parte 3

O estado

A lista de mensagens não é o estado — agora isso é um tipo

```
class Estado(TypedDict):
    mensagens: Annotated[list[AnyMessage], add_messages]
    secoes_prontas: Annotated[list, operator.add]
    voltas: int
```

O **reducer** diz o que fazer quando dois nós escrevem no mesmo campo.

Sem reducer, o campo é SUBSTITUÍDO

Num grafo com nós paralelos, o último que termina sobrescreve os outros.

E nada reclama.

> Sem exceção. Sem log. Sem erro.
>
> O trabalho dos outros ramos desaparece, e o sintoma é um resultado incompleto que **parece correto**.

É o defeito mais silencioso do padrão.

Parte 4

As duas memórias do runtime

Checkpoint e store não são dois nomes para memória

	checkpointer	store
guarda	o estado de **uma** execução	fatos **fora** da execução
endereçado por	`thread_id`	*namespace*
serve para	**retomar**	**lembrar**
escopo	uma conversa	todas

É a fronteira da Aula 08, agora como **duas classes diferentes**.

A prova: a terceira chamada

```
> Oi! Meu nome é Celso e gosto de churrasco.          (conversa-1)
> Qual é o meu nome?                                  (conversa-1)
> Qual é o meu nome, e do que eu gosto de comer?      (conversa-2)  ← thread NOVA

  Seu nome é Celso e você gosta de churrasco.
```

A terceira abre uma thread que **nunca existiu**: o checkpoint está vazio.

Ele acertou porque leu do **store**.

E um detalhe que decide implementação

Checkpointer em memória morre com o processo.

> "Fecha o terminal, reabre, a conversa continua" exige checkpointer em **arquivo ou banco**.
>
> Em memória, a promessa não se cumpre.

Parte 5

O humano no meio do grafo

`interrupt()` suspende a execução DENTRO de um nó

A retomada é uma nova invocação com `Command(resume=...)` — e o valor passado vira o **retorno do** `interrupt()` lá dentro.

Mas **não** continua da linha seguinte: o nó roda **de novo, do começo**.
O que vem antes do `interrupt()` executa duas vezes.

> **Isso só funciona porque existe o checkpointer.**
> A pausa é um checkpoint gravado.

Duas peças independentes produzindo uma terceira capacidade.

Aprovar não é carimbar

```
APROVAÇÃO NECESSÁRIA
  para: alice@exemplo.com.br
  assunto: Reunião de amanhã
  pergunta: Aprova o envio deste e-mail?

Aprova? [s/n] s
Novo assunto (Enter mantém o atual): Reunião de amanhã — confirmando
[enviado] para=alice@exemplo.com.br assunto='Reunião de amanhã — confirmando'
```

O assunto enviado **não é o que o modelo escreveu**. A pessoa aprovou **e corrigiu**.

E aprovar duas vezes não pode enviar dois e-mails: **idempotência**.

Parte 6

O que o framework não dá

A lista — e cada item é reimplantado por vocês

Peça	Por que ela não vem
motivo de término	o nó diz que acabou; não diz **por quê**
detector de laço	`recursion_limit` limita o dano; **não detecta**
orçamento	dos quatro tetos, ele cobre **um**
erro recuperável × fatal	a classificação continua sendo sua

Um agente preso na primeira volta gasta o teto inteiro e devolve o **diagnóstico errado**.

E duas cobranças que nenhum tutorial menciona

A decisão que vira parâmetro com valor padrão. O limiar que vocês justificaram com medição vira um argumento default. O valor fica; a **justificativa sai do código** — e o padrão pode mudar numa atualização.

O paralelismo que não acontece. O grafo despacha em paralelo; o provedor pode atender um por vez.

O paralelismo que não aconteceu — medido

```
[nó] gerar_conto   56.0 s
[nó] gerar_piada   98.0 s
[nó] gerar_poema  175.1 s
total             175.2 s
```

Em paralelo de verdade, os três terminariam por volta dos mesmos 175 s.

Terminaram **um depois do outro**. O total é a **soma**, não o maior.

> "Paralelizei, logo ficou mais rápido" precisa de **medição**, não de diagrama.

O que fica

- **Três produtos**, adotáveis separadamente — e o runtime é o que mais se paga
- Os padrões da Aula 05 **não mudam**: deixam de ser escritos e passam a ser declarados
- O **reducer** é a peça que ninguém lê e que apaga trabalho em silêncio
- **Checkpoint** faz retomar; **store** faz lembrar
- O `interrupt` só existe porque o checkpoint existe
- O framework **não** dá término, detector de laço, orçamento nem classificação de erro

Próxima aula: a ferramenta deixa de ser sua — **MCP**.